

TaskFlow

Dossiers, fichiers et stockage des données

Guide détaillé du dépôt local - inventaire du 8 octobre 2026

Où sont les données ?

TaskFlow conserve ses données métier dans MongoDB. En développement, la commande `npm run db:local` démarre MongoDB WiredTiger et écrit sur disque dans `.local/mongodb/`, à la racine du dépôt. Le chemin est construit dans `scripts/local-mongo.js`. Le dossier est caché (nom commençant par un point) et ignoré par Git dans `.gitignore`; les enregistrements ne sont donc pas commités ni envoyés sur GitHub. Les fichiers `.wt`, journaux et catalogues sont binaires : les ouvrir dans le Finder est possible, mais leur contenu ne se lit pas comme du texte et il ne faut pas les modifier manuellement.

Pour l'inspecter dans le Finder depuis la racine : `open .local/mongodb`. Pour voir les données lisiblement, utiliser l'interface/API ou les commandes d'administration locale de `docs/DONNEES.md`. L'API ne permet pas de lister tous les comptes; la commande diagnostic locale peut afficher les emails, et doit rester réservée à la base de développement. Le mot de passe est stocké sous forme de hash bcrypt. Avec Docker, la persistance est dans le volume `taskflow-data` configuré dans `compose.yaml`. Les tests utilisent des bases temporaires séparées.

Dossiers du projet

Les dossiers générés/ignorés (dépendances, build, caches, données, résultats de test) sont décrits pour expliquer leur usage, mais leurs fichiers internes ne sont pas déroulés un par un. L'inventaire fichier par fichier ci-dessous couvre les fichiers suivis par Git, la documentation Postman locale et les PDF/journaux présents dans `output/pdf`. Le fichier local `backend/.env` est signalé sans en révéler les secrets.

`.github/workflows/` Définition de la CI GitHub Actions.

`.local/mongodb/` Base locale persistante, fichiers binaires WiredTiger ignorés par Git.

`.local/oral-preview/` Rendus temporaires de pages pour contrôler le document oral.

`.local/postman-preview/` Rendus temporaires de pages pour vérifier le guide Postman.

`.local/screenshots/` Captures d'écran générées par les tests navigateur.

`backend/src/config/` Configuration et connexion base de données.

`backend/src/controllers/` Couche HTTP des contrôleurs.

`backend/src/middleware/` Authentification et gestion des erreurs Express.

`backend/src/models/` Schémas Mongoose et documents métier.

`backend/src/routes/` Déclaration des routes API.

`backend/src/services/` Logique métier et accès aux modèles.

`backend/src/utils/` Validation et fonctions d'erreurs partagées.

`backend/test/` Tests du backend.

`docs/oral/` Documents de préparation de l'oral.

docs/postman/ Collection et guides des requêtes API.

e2e/ Tests fonctionnels exécutés dans un navigateur.

frontend/dist/ Build frontend généré, servi par Express en production locale.

frontend/src/components/ Composants React réutilisables.

frontend/src/pages/ Pages de l'application.

output/pdf/ PDF livrables et journaux de compilation.

scripts/ Outils de configuration et de vérification locale.

test-results/ Résultats générés par Playwright.

node_modules/ Dépendances téléchargées par npm ; contenu généré, non détaillé fichier par fichier.

Fichiers, un par un

.github

.github/workflows/ci.yml Décrit les vérifications automatisées lancées par GitHub Actions.

Fichiers à la racine

.gitignore Liste les fichiers locaux ou générés à ne pas versionner, notamment **.local**, **.env**, **node_modules** et **dist**.

README.md Guide principal : démarrage, architecture, routes API, sécurité, bonus et soutenance.

backend

backend/.env.example Modèle des variables de configuration du serveur, sans secret réel.

backend/package.json Dépendances, scripts, configuration Jest et métadonnées du backend.

backend/src/app.js Construit Express, branche les routes, expose santé/OpenAPI/Swagger et sert le frontend compilé.

backend/src/config/db.js Connexion et déconnexion Mongoose.

backend/src/config/env.js Charge et expose PORT, MONGODB_URI et JWT_SECRET.

backend/src/controllers/authController.js Transforme inscription, connexion et changement de mot de passe en réponses HTTP.

backend/src/controllers/taskController.js Gère les réponses HTTP des opérations de liste, création, lecture, modification et suppression.

backend/src/middleware/auth.js Vérifie le Bearer JWT et l'existence du compte avant une route privée.

backend/src/middleware/errors.js Convertit les erreurs applicatives en réponses JSON normalisées.

backend/src/models/Task.js Schéma MongoDB des tâches et forme publique retournée par l'API.

backend/src/models/User.js Schéma MongoDB des utilisateurs ; le hash de mot de passe est caché des lectures ordinaires.

backend/src/routes/authRoutes.js Déclare les routes register, login et changement de mot de passe.

backend/src/routes/taskRoutes.js Déclare les routes CRUD de tâches et impose l'authentification.

backend/src/server.js Valide la configuration, connecte MongoDB, démarre Express et gère l'arrêt propre.

backend/src/services/authService.js Inscrit et authentifie les comptes, hache les mots de passe et crée les JWT.

backend/src/services/taskService.js Valide et stocke les tâches, applique filtres/tri et isole les données par propriétaire.

backend/src/utils/errors.js Fabrique les erreurs HTTP standard de l'API.

backend/src/utils/validation.js Validation stricte des données d'authentification, de tâche, d'identifiant et de filtres.

backend/test/app.test.js Tests d'intégration de l'API : validation, authentification, CRUD, isolation et filtres.

backend/test/env.js Prépare les variables nécessaires à l'environnement de test.

Fichiers à la racine

compose.yaml Configuration Docker Compose de la base MongoDB avec volume persistant.

docs

docs/CONSIGNES_PROF.md Comparaison des consignes pédagogiques avec les choix et fonctionnalités du projet.

docs/SOUTENANCE.md Conseils et déroulé pour préparer la démonstration orale.

docs/openapi.json Contrat OpenAPI consommé par Swagger et décrivant routes, paramètres et réponses.

docs/oral/DISOURS_10_MINUTES.md Texte détaillé proposé pour une présentation orale d'environ dix minutes.

docs/oral/oral_taskflow.tex Source LaTeX du document de préparation à l'oral.

docs/postman/TaskFlow.postman_collection.json Collection Postman importable avec les scénarios de requêtes et scripts de test.

docs/postman/TaskFlow_Postman_requetes.tex Source LaTeX du guide complet des requêtes Postman.

docs/postman/TaskFlow_Postman_requetes.txt Guide texte des requêtes API et séquence de démonstration.

e2e

e2e/taskflow.spec.js Parcours de bout en bout du site dans un navigateur avec Playwright.

Fichiers à la racine

eslint.config.js Règles de lint JavaScript et React du dépôt.

frontend

frontend/index.html Document HTML hôte chargé par Vite avant le code React.

frontend/package.json Dépendances et scripts de développement/build du frontend.

frontend/src/App.jsx Routage React, session utilisateur et accès aux pages.

frontend/src/api.js Fonctions HTTP du navigateur, ajout du jeton et traitement des erreurs API.

frontend/src/components/Account.jsx Profil du compte connecté et formulaire de changement de mot de passe.

frontend/src/components/AccountMenu.jsx Menu utilisateur avec accès au compte et déconnexion.

frontend/src/components/AuthLayout.jsx Mise en page partagée des pages publiques de connexion et inscription.

frontend/src/components/Dashboard.jsx Coordonne chargement des tâches, sélection, création, modification et suppression.

frontend/src/components/Navbar.jsx Navigation principale du site.

frontend/src/components/TaskFilters.jsx Filtres de statut, priorité et échéance, ainsi que contrôles de tri.

frontend/src/components/TaskForm.jsx Formulaire de création et modification d'une tâche.

frontend/src/components/TaskItem.jsx Affiche une tâche et ses actions.

frontend/src/components/TaskList.jsx Affiche la liste de tâches et son état vide.

frontend/src/index.css Styles globaux et présentation responsive.

frontend/src/main.jsx Point d'entrée React ; monte l'application dans la page HTML.

frontend/src/pages/LoginPage.jsx Formulaire et flux de connexion.

frontend/src/pages/Registerpage.jsx Formulaire et flux de création de compte.

frontend/src/taskUtils.js Libellés, formatage des dates et fonctions utilitaires d'affichage des tâches.

frontend/vite.config.js Configuration Vite, serveur frontend et proxy des appels /api vers Express.

output

output/pdf/TaskFlow_Postman_requetes.log Journal de compilation LaTeX du guide Postman.

output/pdf/TaskFlow_Postman_requetes.pdf PDF compilé du guide des requêtes Postman.

output/pdf/oral_taskflow.log Journal de compilation LaTeX du PDF oral.

output/pdf/oral_taskflow.pdf PDF compilé du document de préparation à la soutenance.

Fichiers à la racine

package-lock.json Verrouille les versions exactes des dépendances npm à installer.

package.json Scripts racine, workspaces npm et outils partagés du projet.

playwright.config.js Configuration des navigateurs, du serveur et des suites Playwright.

scripts

scripts/check-persistence.js Vérifie qu'une tâche survit à l'arrêt puis au redémarrage du serveur.

scripts/e2e-server.js Prépare et arrête l'environnement serveur/base isolé des tests navigateur.

scripts/local-mongo.js Démarre MongoDB local persistant dans `.local/mongodb` avec WiredTiger.

scripts/setup.js Initialise la configuration locale du backend sans remplacer un fichier déjà présent.

Périmètre et fichiers générés

L'inventaire est basé sur les fichiers présents dans le dépôt au moment de la rédaction. Les dossiers `node_modules/` contiennent les bibliothèques téléchargées par npm et changent selon l'installation ; ils ne sont pas détaillés fichier par fichier. `frontend/dist/`, `test-results/`, les captures et prévisualisations sont générés par les commandes de build ou de test. `.local/mongodb/` est une vraie base persistante : ne pas supprimer ce dossier si l'on veut garder les comptes et tâches.

Pour les commandes d'inspection et les détails sur les collections MongoDB, voir `docs/DONNEES.md`. Pour le contrat complet de l'API, voir `docs/openapi.json` ou ouvrir Swagger à `http://localhost:3000/api/docs/` après démarrage.